

Lab 3: Programming a Real Processing-in-Memory Architecture

Introduction

In this lab, I performed an analysis of a real Processing-in-Memory (PIM) architecture. This involved implementing and evaluating four distinct tasks for the UPMEM PIM System: characterizing data transfer performance, scaling the AXPY vector operation, assessing performance across different data types and operations, and exploring intra-DPU synchronization primitives for parallel vector reduction.

Task 1 Transferring Data between Main Memory and PIM-enabled Memory

This task was concerned with the data transfer bandwidth between the host CPU and the DPUs. Aside from computation inputs and outputs being transferred between the host CPU and the DPUs, communication between DPUs has also to be routed through the host as an intermediary. Therefore having high transfer bandwidth is important for the PIM system to be effective. Documentation for runtime communication primitives is available in the UPMEM manual.

Implementation Details

There are three different data transfer methods: `SERIAL`, `PARALLEL` and `BROADCAST`. Data transfers have to be invoked on the host, regardless of the transfer direction (CPU to DPU or DPU to CPU). In contrast to the former two methods, `BROADCAST` can only unidirectionally distribute data from the host to the DPUs. The `SERIAL` and `PARALLEL` methods rely on the `dpu_prepare_xfer` function defined

This

```
#ifdef SERIAL    // Serial transfers
    DPU_ASSERT(dpu_push_xfer(
        dpu_set,                                // the identifier of the DPU set
        DPU_XFER_TO_DPU,                        // direction of the transfer
        DPU_MRAM_HEAP_POINTER_NAME,             // location of symbol where to place data
        0,                                       // byte offset from symbol address
        input_size_dpu_8bytes * sizeof(T),     // number of bytes to copy
        DPU_XFER_DEFAULT                        // options of the transfer
    ));

#elif BROADCAST // Broadcast transfers
    DPU_ASSERT(dpu_broadcast_to(
        dpu_set,
        DPU_MRAM_HEAP_POINTER_NAME,
        0,
        bufferX,
        input_size_dpu_8bytes * sizeof(T),
        DPU_XFER_DEFAULT
    ));

#else // Parallel transfers
    DPU_ASSERT(dpu_push_xfer(
        dpu_set,
        DPU_XFER_TO_DPU,
        DPU_MRAM_HEAP_POINTER_NAME,
        0,
        input_size_dpu_8bytes * sizeof(T),
        DPU_XFER_ASYNC
    ));

    // Wait for the end of the execution on the DPU set.
    DPU_ASSERT(dpu_sync(dpu_set));
#endif
```

Evaluation

Analysis and Observations

Task 2 AXPY

Implementation Details

Evaluation

Analysis and Observations

Task 3 Operations and Data Types

Implementation Details

Evaluation

Analysis and Observations

Task 4 Vector Reduction

Implementation Details

Evaluation

Analysis and Observations

Citations

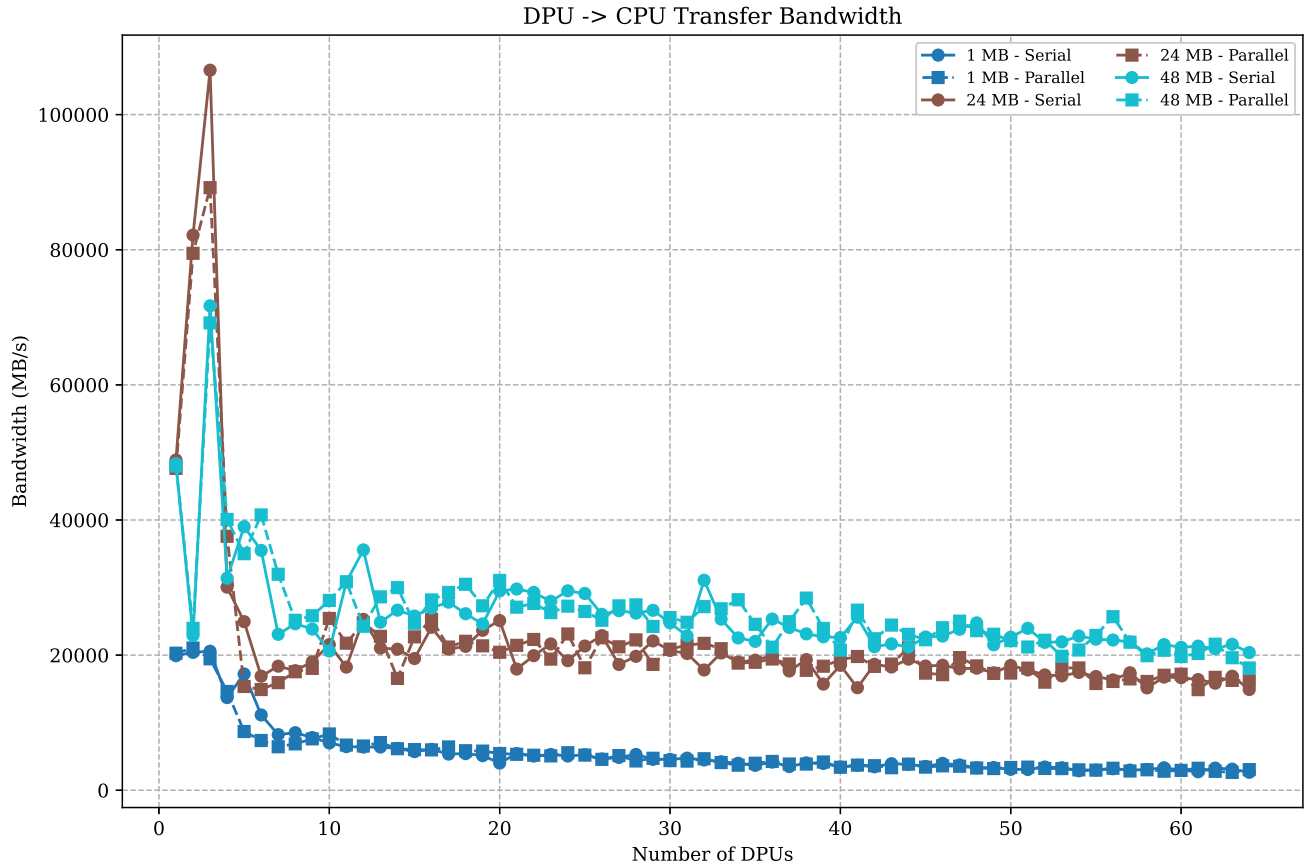
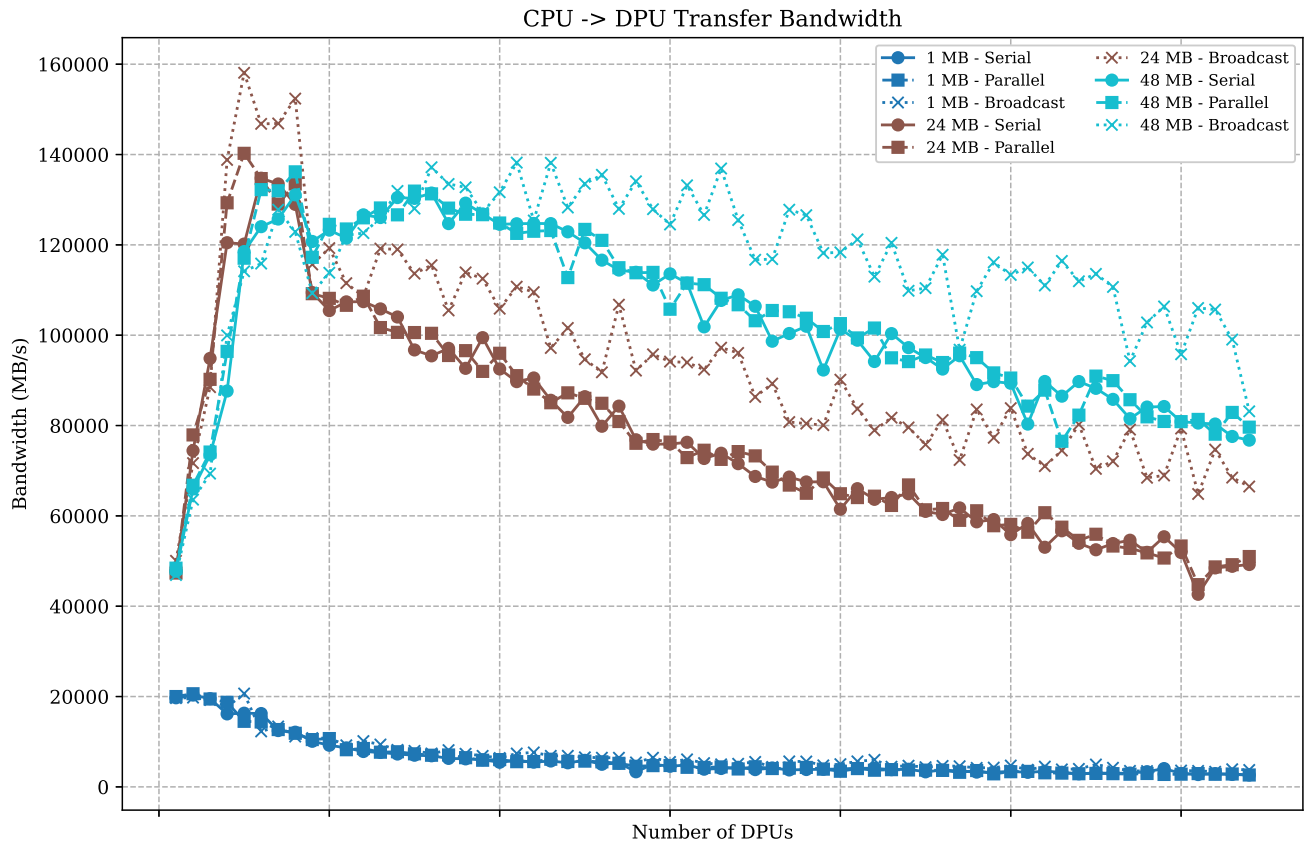


Figure 1: Transfer Bandwidth